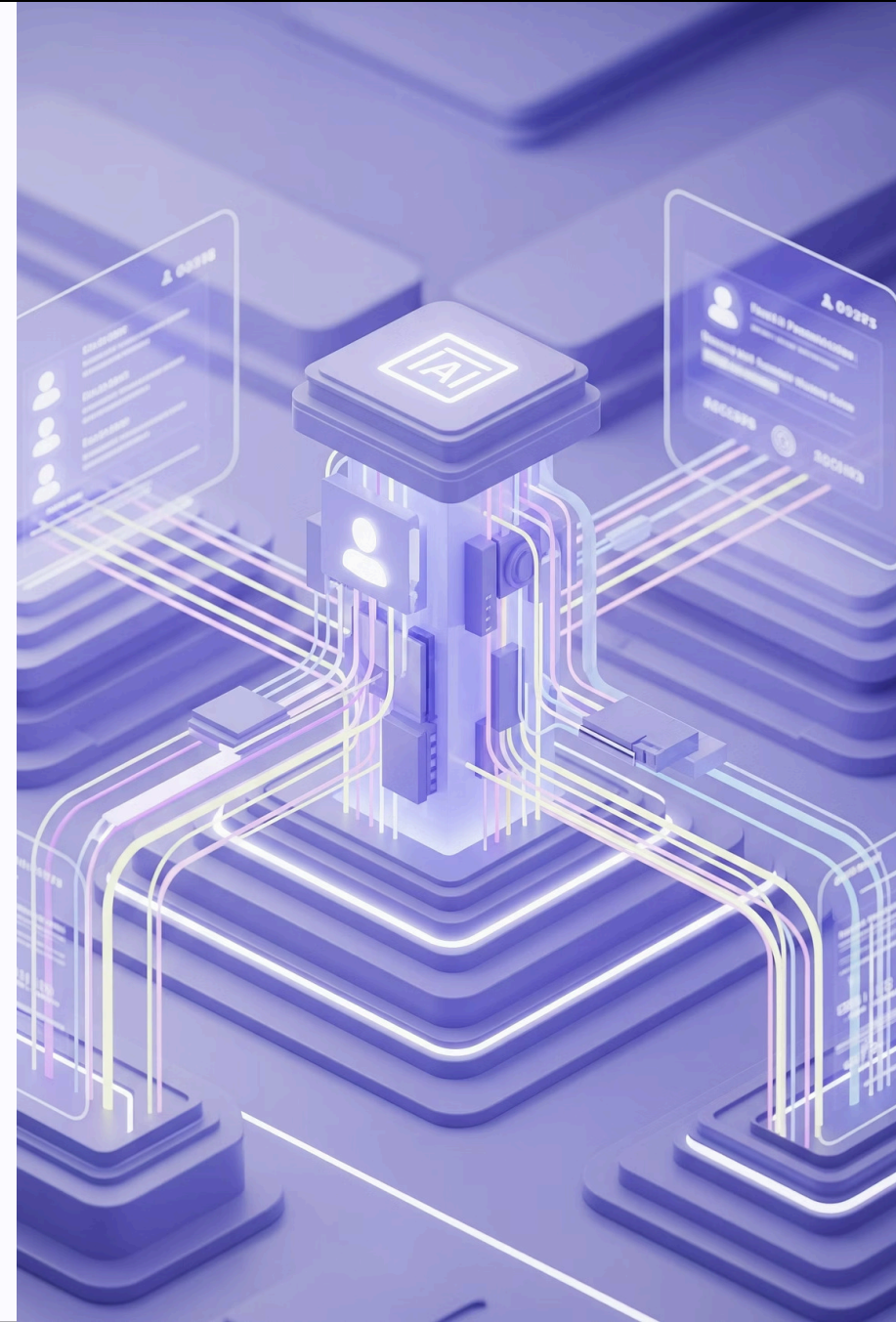
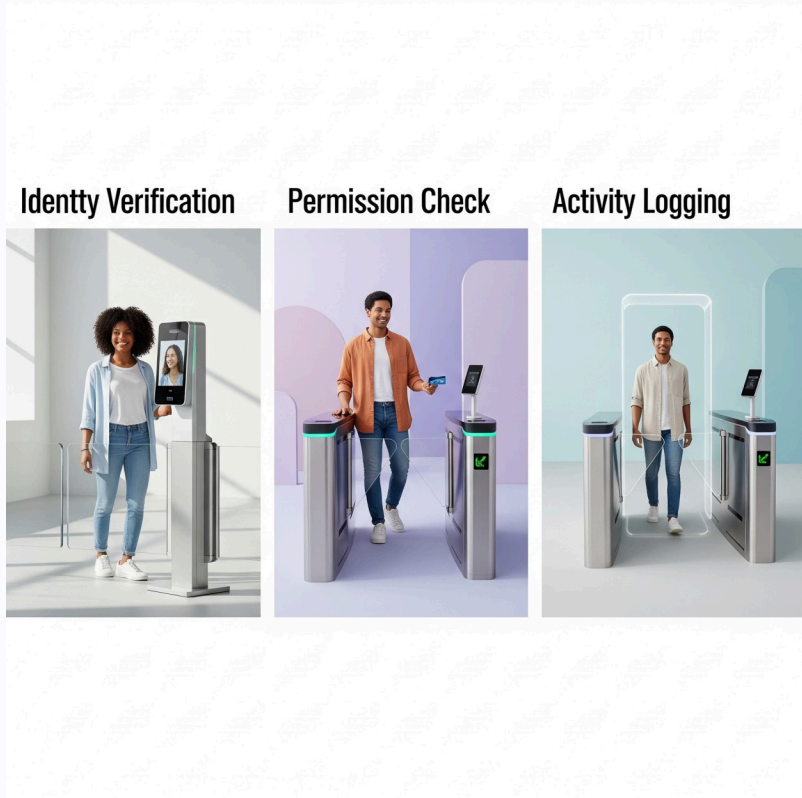


Role-Based Access Control (RBAC) for AI Infrastructure

A comprehensive framework for controlling who can do **what** on **which** resource, establishing the foundation for **multi-tenant security** and mapping organizational roles to least-privilege permissions.



Authentication vs. Authorization



Authentication (AuthN)

Verifies **who you are** through SSO, OIDC, MFA

Authorization (AuthZ)

Determines **what you can do** through RBAC

Accounting (Audit)

Records **what happened** through logs

RBAC forms the core of authorization in clusters, clouds, and MLOps tools

Why RBAC in AI Infrastructure?

Protect Valuable Assets

Safeguard **datasets** containing PII/PHI, **models** with IP value, and **clusters** with expensive GPU resources

Enable Multi-Tenant Sharing

Allow multiple teams to safely share infrastructure while maintaining isolation and security boundaries

Satisfy Compliance Requirements

Meet regulatory demands from SOC 2, HIPAA, GDPR, and other frameworks

Prevent Costly Mistakes

Distinguish between permissions like "train" and "delete dataset" to avoid accidental data loss or exposure

RBAC vs ABAC vs ReBAC

RBAC: Role-Based Access Control

Maps roles to permissions

- Simple and scalable
- Easy to understand
- Common starting point

ABAC: Attribute-Based Access Control

Uses attributes to make decisions

- Environment: prod vs. dev
- Data classification: sensitive
- Fine-grained control

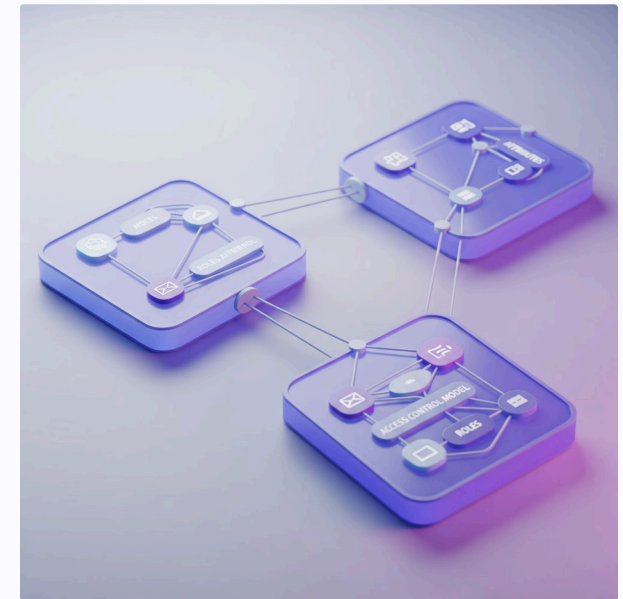
ReBAC: Relationship-Based Access Control

Uses ownership graphs

- Based on relationships
- Useful for data lineage
- Complex but powerful

Recommended Approach

Start with RBAC, then add ABAC conditions for sensitive data and operations as your security needs mature.



Least Privilege & Segregation of Duties

Core Principles

Minimum Necessary Rights

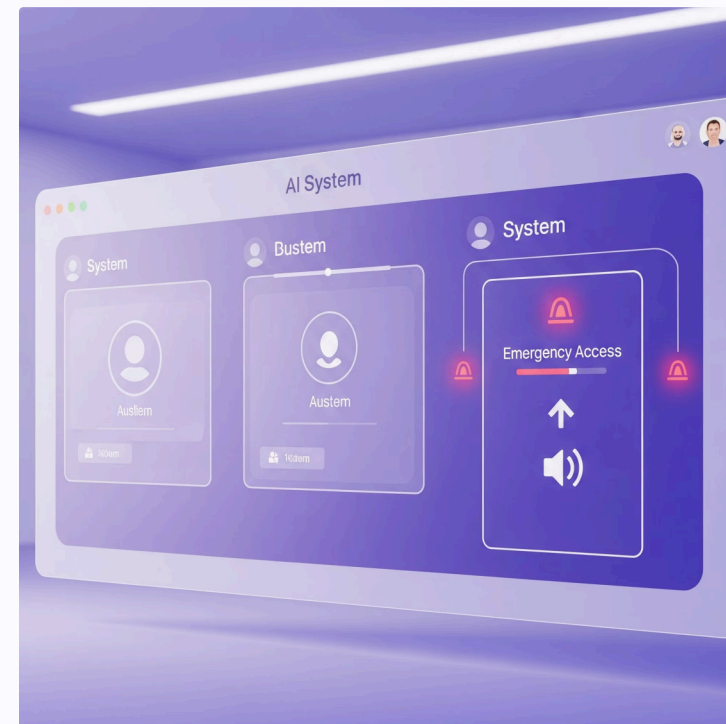
Grant only the permissions required to perform specific job functions—nothing more

Break-Glass Access

Implement temporary permission elevation with tight audit controls for emergencies

Separate Duties

Maintain clear boundaries between data engineers, ML researchers, and platform administrators



Segregation ensures that no single role can compromise the entire system, reducing risk and providing crucial audit boundaries.

Common AI Roles & Scopes



Reader

View datasets, metrics, dashboards

Read-only access to non-sensitive information



Contributor

Run experiments, read non-sensitive data

Limited write access within sandboxed environments



Trainer

Submit/train jobs, read/write to experiment buckets

Access to compute resources and specific datasets



Pusher/Deployer

Create model versions, canary deploy

Ability to promote models to staging/testing



Platform Admin

Cluster & network changes (no data by default)

Infrastructure access without automatic data access



Data Steward

Governs sensitive tables & approvals

Manages access controls and compliance for sensitive data

Kubernetes RBAC: Core Concepts

Building Blocks

→ Subject

User, Group, or ServiceAccount that needs access

→ Role/ClusterRole

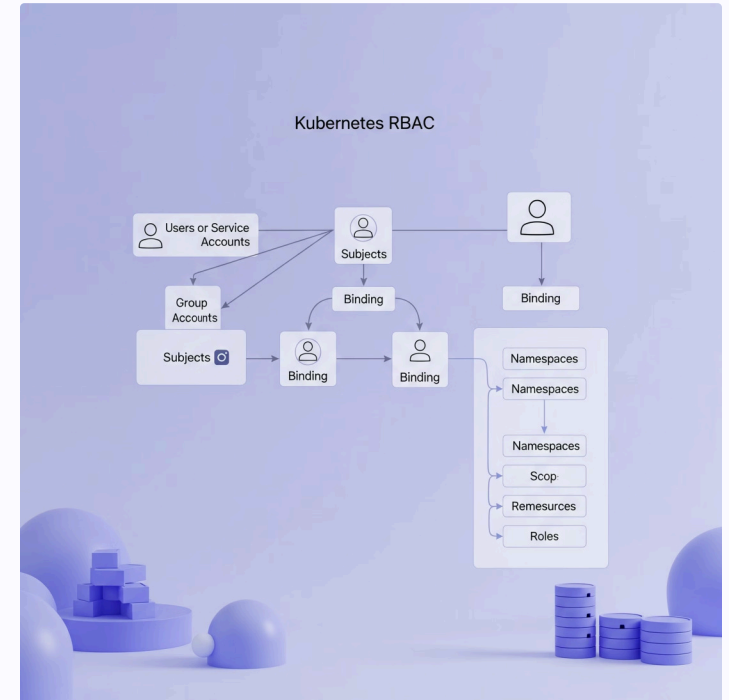
Defines allowed verbs on resources

→ RoleBinding/ClusterRoleBinding

Attaches roles to subjects

Scoping Options

- **Namespace:** Isolate teams and applications
- **ClusterRole:** Define permissions across the cluster



Kubernetes RBAC provides granular control over who can access what resources, using what actions, and in which parts of the cluster.

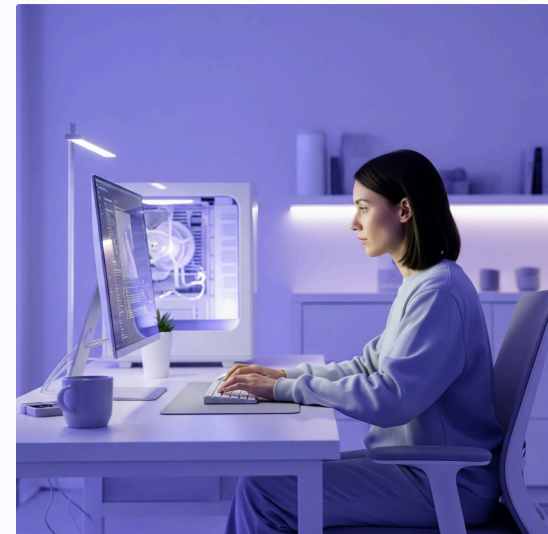
K8s Example: Namespace-Scoped Role

This example creates a role that allows Alice to submit jobs and read logs, but only within the team-a namespace.

```
# role-ml-trainer.yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: team-a
  name: ml-trainer
rules:
- apiGroups: ["batch"]
  resources: ["jobs"]
  verbs: ["create", "get", "list", "watch"]
- apiGroups: [""]
  resources: ["pods", "pods/log"]
  verbs: ["get", "list", "watch"]
---
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: bind-ml-trainer
  namespace: team-a
subjects:
- kind: User
  name: alice@company.com
roleRef:
  kind: Role
  name: ml-trainer
  apiGroup: rbac.authorization.k8s.io
```

What This Allows

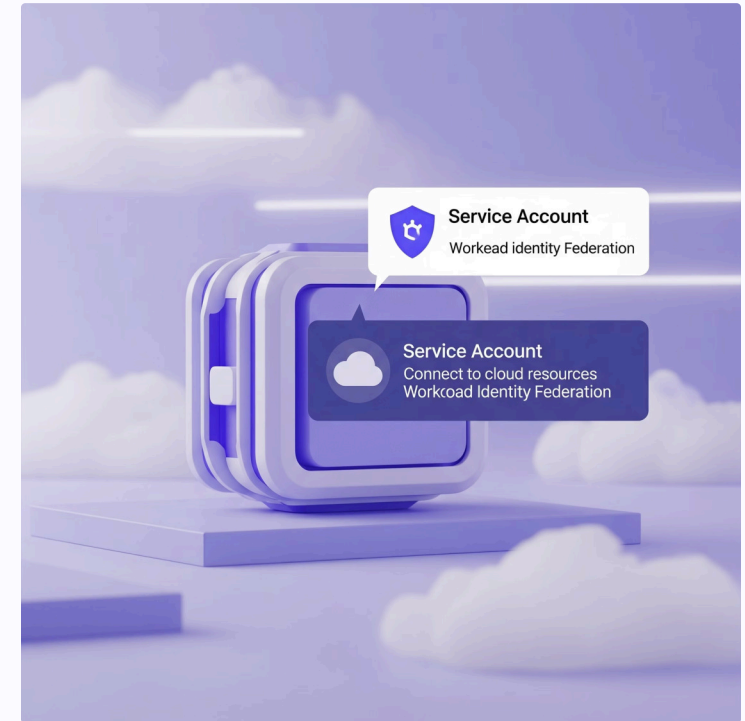
- Submit batch jobs
- View job status
- Read pod logs
- All scoped to team-a only



Service Accounts & Workload Identity

Best Practices

- Use **ServiceAccounts** for jobs and pods instead of human credentials
- Bind ServiceAccounts to cloud IAM using platform-specific features:
 - GKE Workload Identity
 - EKS IAM Roles for Service Accounts (IRSA)
- Secure access to external resources without storing static credentials:
 - Object stores (S3, GCS)
 - Feature stores
 - Container registries



Workload Identity creates a secure, manageable way for containerized applications to access cloud services without embedded credentials.

Cloud IAM: AWS Example (EKS + S3)

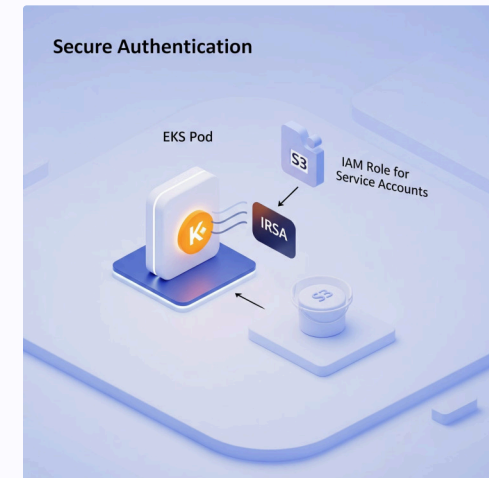
```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": [
      "s3:GetObject",
      "s3:PutObject"
    ],
    "Resource": [
      "arn:aws:s3:::ml-team-a/*"
    ]
  }]
}
```

This policy grants read/write access specifically to the team-a bucket prefix, avoiding broad permissions.

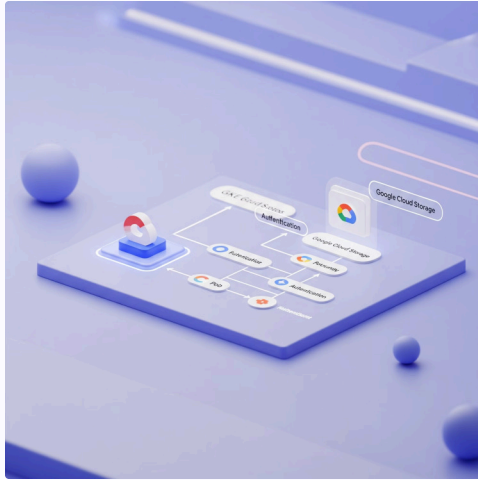
Implementation

Attach this policy to an **IAM Role for Service Account (IRSA)** used by Team-A's training workloads to:

- Restrict access to only the team's data
- Avoid storing AWS credentials in pods
- Enable automatic credential rotation
- Maintain audit trail of all access



Cloud IAM: GCP Example (GKE + GCS)



Implementation Steps

1. Enable Workload Identity on GKE cluster
2. Create a Google Service Account (GSA)
3. Grant [Storage Object Admin](#) on `gs://ml-team-a/*` to the GSA
4. Create a Kubernetes ServiceAccount (KSA) in team's namespace
5. Bind the KSA to the GSA using IAM policy
6. Use the KSA in pod specifications

❌ Avoid Overprivileged Access

Never grant project-level permissions to workloads. Always scope to specific buckets or prefixes.

✅ Benefits

- Short-lived OAuth tokens
- No static keys in code
- Fine-grained bucket access
- Full audit trail

Data Layer RBAC

Data Warehouse (BigQuery/Snowflake)

- Schema and table-level grants
- Column-level security for sensitive fields
- Row-level security for multi-tenant data

Feature Store (Feast/Tecton)

- Project-level isolation
- Feature-view specific permissions
- Separate online/offline access controls

Access Patterns



Training Data Access

Read permissions on relevant tables/features



Offline Materialization

Limited write permissions for preprocessing



Online Serving

Read-only access to online store for inference

Model Registry & Serving RBAC

Model Registry

Systems like MLflow, Vertex AI, and SageMaker Registry require distinct roles:



Viewer

Read-only access to model metadata and metrics



Model-Writer

Register new models and versions



Approver

Review and transition models between stages



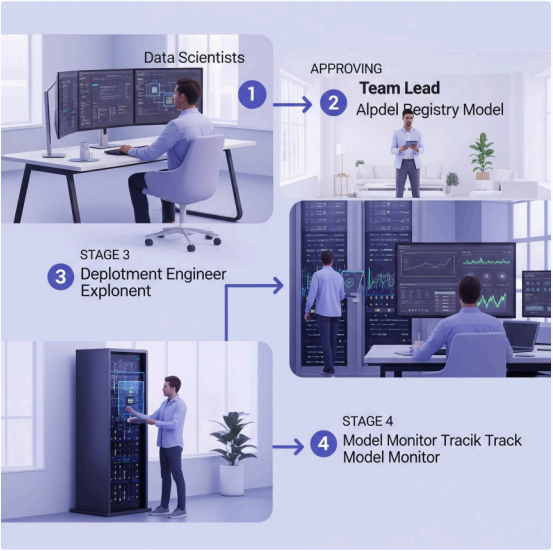
Deployer

Deploy approved models to serving infrastructure

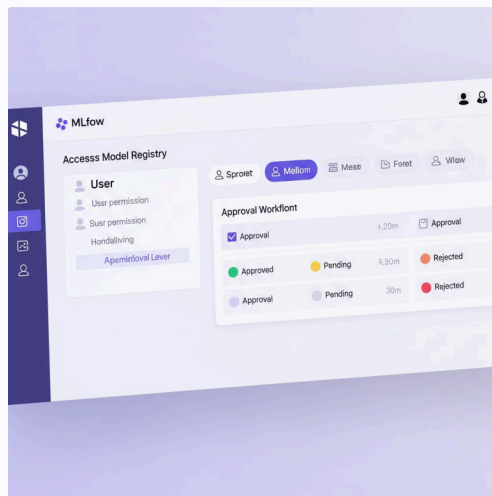
Model Serving

For systems like KFServing/Triton/SageMaker:

- Allow deployers to create new **Revisions**
- Restrict **traffic shifts** to approvers
- Separate permissions for:
 - Creating endpoints
 - Updating model versions
 - Controlling traffic distribution
 - Viewing serving metrics



Example: MLflow Permissions (conceptual)



`mlflow.models:view`

Read-only access to view models and their metadata

Assigned to: Everyone in the organization

`mlflow.models:write`

Ability to register new models and create versions

Assigned to: Team maintainers and model developers

`mlflow.models:transition_stage`

Authority to move models between stages (Dev→Staging→Prod)

Assigned to: Designated approvers only

`mlflow.artifacts:read`

Access to model binaries and artifacts for deployment

Assigned to: CI/CD runner service accounts

CI/CD & Environments

Environment Separation

Maintain distinct roles and permissions per environment:

Development

Permissive for experimentation but isolated from other environments

Staging

Testing with production-like data but stricter controls

Production

Highly restricted access with explicit approvals

CI/CD Security

- Pipelines **temporarily assume deploy roles** only during rollout
- Use OIDC-backed authentication for CI/CD systems
- Enforce strict boundaries between environments
- Prevent development jobs from accessing production datasets
- Log all cross-environment promotions



Policy as Code (OPA/Gatekeeper, Kyverno)

Enforce Guardrails at Admission

```
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sPSPPrivilegedContainer
metadata:
  name: prevent-privileged-containers
spec:
  match:
    kinds:
      - apiGroups: [""]
        kinds: ["Pod"]
    excludedNamespaces: ["kube-system"]
  parameters: {}
```

Key Policies for AI Infrastructure

- No hostPath mounts or privileged pods
- Only allow images from trusted registries
- Enforce namespace-scoped GPU quotas
- Require resource limits on all containers
- Mandate security contexts and non-root users



Implementation Best Practices

- Version policies in Git alongside infrastructure
- Test policies in CI to prevent breaking changes
- Start with audit mode before enforcement
- Create exceptions process for legitimate needs
- Monitor policy violations to identify patterns

Secrets & Keys

→ Use Cloud-Native Secret Stores

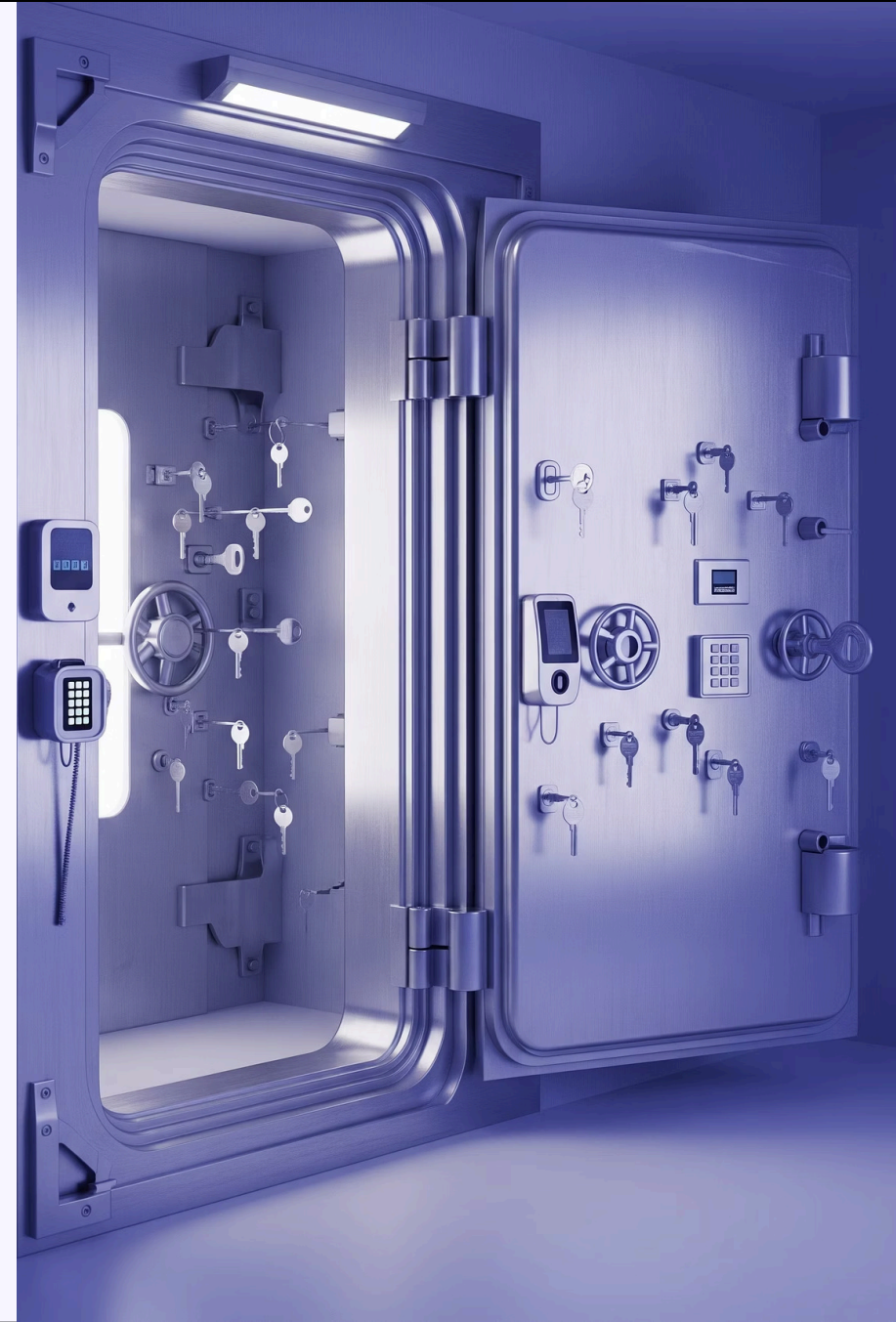
Store secrets in [cloud KMS/Secrets Manager](#); mount via CSI drivers rather than environment variables or configuration files

→ Prevent Plaintext Secrets

Enforce policies that deny plaintext secrets in environment variables or unencrypted configuration files

→ Implement Key Rotation

Regularly rotate keys and use short-lived tokens (STS) to limit the blast radius of credential compromise



Auditing & Monitoring

Enable Comprehensive Logging

- **Kubernetes Audit Logs:** Track all API server requests
- **Cloud Audit Logs:** Record all control plane operations
- **CloudTrail:** Log AWS API calls and activities
- **Data Access Logs:** Monitor who accessed sensitive data

Create Per-Tenant Dashboards

Build visibility into:

- Who ran what workloads
- Which data was accessed when
- Resource consumption patterns

Critical Alerts

Privilege Escalation

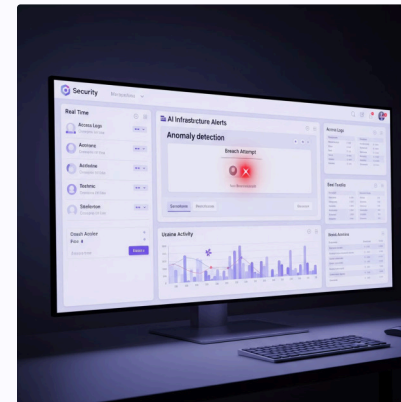
Detect when users gain increased permissions

Mass Data Reads

Flag unusual data access patterns

Unusual Model Downloads

Identify potential IP theft attempts



JIT Access & Break-Glass

Just-In-Time Access

Request Elevated Access

Engineer submits ticket with justification and time window

Approval Workflow

Manager or security team reviews and approves request

Temporary Credentials Issued

Time-bounded elevated permissions granted

Enhanced Logging

All actions during elevated session recorded

Auto-Revocation

Permissions automatically expire after time window

Break-Glass Procedures

For emergency incident response:

- Maintain separate emergency credentials
- Store with physical or digital safeguards
- Require pager-duty workflow for access
- Always use multi-person authentication
- Record all actions in immutable audit log
- Conduct post-incident review



Mapping to Compliance

SOC 2 CC6/CC7

Logical access controls, change management, and security monitoring

- RBAC provides evidence of logical access implementation
- Role transitions map to change management controls

HIPAA

Minimum necessary access and audit controls

- Least privilege principle directly supports minimum necessary
- Audit logs demonstrate access review capabilities

GDPR

Data minimization and purpose limitation

- Scoped permissions enforce purpose limitation
- JIT access ensures appropriate data use



RBAC as a Control

RBAC itself is an **implementation of controls**. Your logs and approval workflows provide the **evidence of enforcement** needed for audits.

Map specific roles and permissions to compliance requirements in your documentation to streamline audit processes.

Multi-Tenant Patterns that Work

Per-Team Namespaces/Projects

Isolate teams with dedicated Kubernetes namespaces or cloud projects

- Apply resource quotas to prevent resource monopolization
- Enforce network policies to limit cross-team communication

Per-Tenant Buckets/Prefixes

Separate storage for datasets and artifacts

- Use dedicated S3/GCS buckets or prefixes
- Apply bucket policies that enforce tenant isolation

Per-Service Account Identities

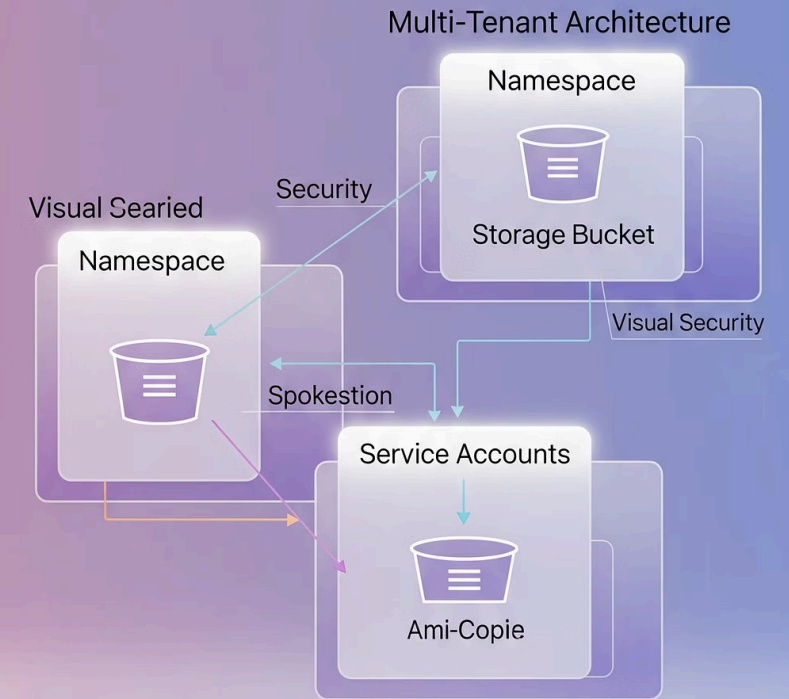
Unique identities for each workload type

- Dedicated service accounts per tenant and function
- Workload identity federation to cloud resources

Central Model Registry with Approval

Shared registry with strong access controls

- Stage-based promotion with approvals
- Tenant-specific views and permissions





Anti-Patterns & Pitfalls

"Everyone is Admin" in Dev

Overly permissive development environments create bad habits that leak into production. Maintain consistent security posture across all environments.

Shared Long-Lived Cloud Keys

Using static API keys in notebooks or code creates unauditible access and significant risk. Use ephemeral credentials tied to identity instead.

Broad Wildcard Policies

Policies like "s3:*" on data stores allow excessive access. Create fine-grained policies that grant only specific actions on specific resources.

Over-Privileged CI Runners

CI systems with persistent admin credentials are high-value targets. Use just-in-time, scoped permissions for deployment pipelines.

Rollout Checklist

Implementation Steps

01

Define Roles

Map to organizational groups in your Identity Provider (IdP)

02

Create Resource Boundaries

Set up namespaces, buckets, and bind appropriate roles

03

Secure Service Identities

Lock down service accounts and configure workload identity

04

Implement Guardrails

Add admission policies, secrets management, and audit logging

05

Test Boundary Cases

Verify that a trainee cannot delete production models

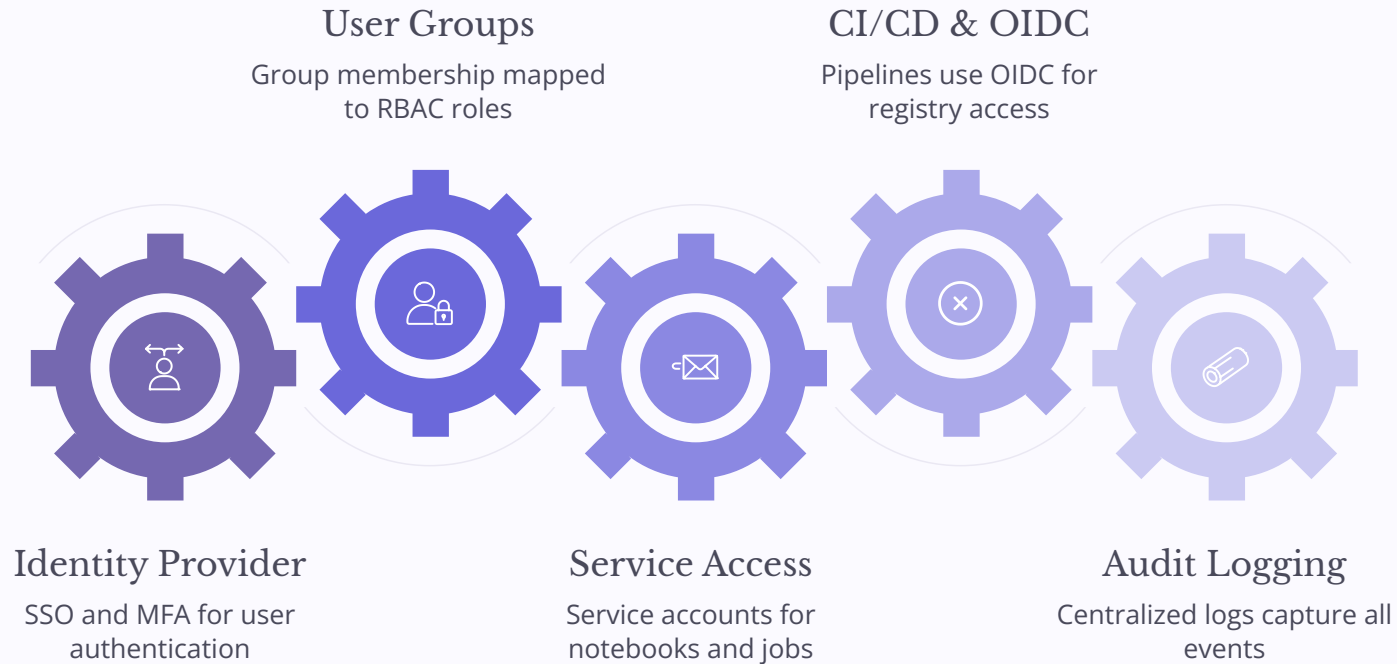


Start Small, Iterate Often

Begin with a minimal viable security model and gradually enhance it. Trying to implement everything at once often leads to failure or excessive friction.

Document your RBAC model and create onboarding guides for each role to ensure smooth adoption by your teams.

End-to-End Flow



This end-to-end security flow ensures that identity and permissions are consistently managed from authentication through to resource access, with comprehensive audit trails throughout the system.

Summary: RBAC for AI Infrastructure

Key Takeaways

→ Least Privilege Foundation

RBAC enforces appropriate access boundaries across compute, data, and models

→ Compliance Enabler

Essential infrastructure for secure, compliant multi-tenant AI operations

→ Integrated Security Model

Combine with workload identity, policy-as-code, and comprehensive auditing

Next Steps

1. Audit current access patterns
2. Define role hierarchy for your organization
3. Implement progressive security improvements
4. Train teams on secure access patterns



✔ The RBAC Advantage

Properly implemented RBAC allows you to safely share expensive AI infrastructure while maintaining security, auditability, and compliance—enabling innovation without compromising control.